

```
In [1]: import math
from dataclasses import dataclass
from typing import List, Dict

# -----
# Helper math functions
# -----
def sigmoid(x: float) -> float:
    return 1 / (1 + math.exp(-x))

def tanh(x: float) -> float:
    return math.tanh(x)

def line(char="=", width=70):
    print(char * width)

def section(title: str):
    print("\n")
    line("=")
    print(f" {title}")
    line("=")

def show_formula(label: str, expr: str, value: float, decimals: int = 3):
    print(f'{label:<18} {expr} = {value:.{decimals}f}')
```

```
In [2]: @dataclass
class LSTMParams:
    Wf: float; Whf: float; bf: float
    Wi: float; Whi: float; bi: float
    Wc: float; Whc: float; bc: float
    Wo: float; Who: float; bo: float

@dataclass
class OutputLayer:
    Wy: float; by: float

@dataclass
class LSTMState:
    h: float = 0.0
    c: float = 0.0
```

```
In [3]: def lstm_step(x_t: float, prev_state: LSTMState, params: LSTMParams) -> Dict[str, float]:
    h_prev = prev_state.h
    c_prev = prev_state.c

    # Gates
    f_t_input = params.Wf * x_t + params.Whf * h_prev + params.bf
    i_t_input = params.Wi * x_t + params.Whi * h_prev + params.bi
    c_hat_input = params.Wc * x_t + params.Whc * h_prev + params.bc
    o_t_input = params.Wo * x_t + params.Who * h_prev + params.bo

    f_t = sigmoid(f_t_input)
    i_t = sigmoid(i_t_input)
    c_hat_t = tanh(c_hat_input)
    c_t = f_t * c_prev + i_t * c_hat_t
    o_t = sigmoid(o_t_input)
    h_t = o_t * tanh(c_t)

    return {
        "x_t": x_t, "h_prev": h_prev, "c_prev": c_prev, "f_t": f_t,
        "i_t": i_t, "c_hat_t": c_hat_t, "c_t": c_t, "o_t": o_t, "h_t": h_t
    }
```

```
In [4]: def print_time_step(t: int, result: Dict[str, float]):
    x_t = result["x_t"]; h_prev = result["h_prev"]; c_prev = result["c_prev"]
    print(f"\n⦿ Time Step t = {t}, Input x_{t} = {x_t}")
    line("-")
    show_formula("1) Forget gate:", f"o(0.5*[x_t] + 0.1*[h_prev:.3f] + 0)", result["f_t"])
    show_formula("2) Input gate:", f"o(0.6*[x_t] + 0.2*[h_prev:.3f] + 0)", result["i_t"])
    show_formula("3) Candidate:", f"tanh(0.7*[x_t] + 0.3*[h_prev:.3f] + 0)", result["c_hat_t"])
    c_new_from_old = result["f_t"] * c_prev
    c_new_from_candidate = result["i_t"] * result["c_hat_t"]
    print(f"4) Cell state: ({result['f_t']:.3f} * {c_prev:.3f}) + ({result['i_t']:.3f} * {result['c_hat_t']:.3f}) = {result['c_t']:.3f}")
    show_formula("5) Output gate:", f"o(0.8*[x_t] + 0.4*[h_prev:.3f] + 0)", result["o_t"])
    print(f"6) Hidden state: {result['o_t']:.3f} * tanh({result['c_t']:.3f}) = {result['h_t']:.3f}")

def run_lstm_example():
    section("LSTM NUMERICAL EXAMPLE")
    X = [1, 2, 3]
    params = LSTMParams(Wf=0.5, Whf=0.1, bf=0.0, Wi=0.6, Whi=0.2, bi=0.0, Wc=0.7, Whc=0.3, bc=0.0, Wo=0.8, Who=0.4, bo=0.0)
    output_layer = OutputLayer(Wy=4.0, by=0.0)
    state = LSTMState(h=0.0, c=0.0)

    history = []
    for t, x_t in enumerate(X, start=1):
        result = lstm_step(x_t, state, params)
        print_time_step(t, result)
        state = LSTMState(h=result["h_t"], c=result["c_t"])
        history.append(result)

    section("STEP 2 - PREDICT THE NEXT VALUE")
    result_t4 = lstm_step(4, state, params)
    print_time_step(4, result_t4)
    y_hat = output_layer.Wy * result_t4["h_t"] + output_layer.by
    print(f"\nFinal Prediction: {y_hat:.3f}")

if __name__ == "__main__":
    run_lstm_example()
```

=====

LSTM NUMERICAL EXAMPLE

=====

⦿ Time Step t = 1, Input x₁ = 1

1) Forget gate: $o(0.5 * 1 + 0.1 * 0.00 + 0) = 0.500$

```
1) Forget gate:  $\sigma(0.5 \cdot 1 + 0.1 \cdot 0.000 + 0) = 0.622$ 
2) Input gate:  $\sigma(0.6 \cdot 1 + 0.2 \cdot 0.000 + 0) = 0.646$ 
3) Candidate:  $\tanh(0.7 \cdot 1 + 0.3 \cdot 0.000 + 0) = 0.604$ 
4) Cell state:  $(0.622 \cdot 0.000) + (0.646 \cdot 0.604) = 0.390$ 
5) Output gate:  $\sigma(0.8 \cdot 1 + 0.4 \cdot 0.000 + 0) = 0.690$ 
6) Hidden state:  $0.690 \cdot \tanh(0.390) = 0.256$ 

● Time Step t = 2, Input x_2 = 2
-----
1) Forget gate:  $\sigma(0.5 \cdot 2 + 0.1 \cdot 0.256 + 0) = 0.736$ 
2) Input gate:  $\sigma(0.6 \cdot 2 + 0.2 \cdot 0.256 + 0) = 0.778$ 
3) Candidate:  $\tanh(0.7 \cdot 2 + 0.3 \cdot 0.256 + 0) = 0.901$ 
4) Cell state:  $(0.736 \cdot 0.390) + (0.778 \cdot 0.901) = 0.988$ 
5) Output gate:  $\sigma(0.8 \cdot 2 + 0.4 \cdot 0.256 + 0) = 0.846$ 
6) Hidden state:  $0.846 \cdot \tanh(0.988) = 0.640$ 

● Time Step t = 3, Input x_3 = 3
-----
1) Forget gate:  $\sigma(0.5 \cdot 3 + 0.1 \cdot 0.640 + 0) = 0.827$ 
2) Input gate:  $\sigma(0.6 \cdot 3 + 0.2 \cdot 0.640 + 0) = 0.873$ 
3) Candidate:  $\tanh(0.7 \cdot 3 + 0.3 \cdot 0.640 + 0) = 0.980$ 
4) Cell state:  $(0.827 \cdot 0.988) + (0.873 \cdot 0.980) = 1.672$ 
5) Output gate:  $\sigma(0.8 \cdot 3 + 0.4 \cdot 0.640 + 0) = 0.934$ 
6) Hidden state:  $0.934 \cdot \tanh(1.672) = 0.871$ 

=====
STEP 2 - PREDICT THE NEXT VALUE
=====

● Time Step t = 4, Input x_4 = 4
-----
1) Forget gate:  $\sigma(0.5 \cdot 4 + 0.1 \cdot 0.871 + 0) = 0.890$ 
2) Input gate:  $\sigma(0.6 \cdot 4 + 0.2 \cdot 0.871 + 0) = 0.929$ 
3) Candidate:  $\tanh(0.7 \cdot 4 + 0.3 \cdot 0.871 + 0) = 0.996$ 
4) Cell state:  $(0.890 \cdot 1.672) + (0.929 \cdot 0.996) = 2.413$ 
5) Output gate:  $\sigma(0.8 \cdot 4 + 0.4 \cdot 0.871 + 0) = 0.972$ 
6) Hidden state:  $0.972 \cdot \tanh(2.413) = 0.957$ 

Final Prediction: 3.826
```